From First Principles to the First Step of Dobbertin's Theorem 1

A bottom-up account of the `DobbertinStep1` Lean 4 formalisation

Abstract

We reconstruct, *from the ground up*, the Lean 4 / Mathlib formalisation of the opening step of the proof of Theorem 1 in H. Dobbertin, *Kasami Power Functions, Permutation Polynomials and Cyclic Difference Sets* (NATO Sci. Ser. C **542**, 1999, pp. 133–158): the implication *equation (1)* $\Rightarrow$ *equation (2)*. Starting from the simplest foundational ingredients (a handful of characteristic-2 finite-field facts from Mathlib), we introduce the definitions, then compose progressively richer lemmas, each presented in ordinary mathematical notation *beside* its exact Lean statement, until the two ingredients —an Artin–Schreier telescoping and the fact that the trace is a bit— combine into the headline theorem. A layered build diagram, a commutative diagram of the derivation, and dependency graphs make the compositional structure explicit.

Contents

1 The target, and the plan

Fix $F = \mathbb{F}_{2^n}$, a finite field of characteristic 2, and integers k, k' with $k\,k' \equiv 1 \pmod{n}$. For $c \in F$, Dobbertin's equation (1) (cleared of denominators) reads

$$c\,x^{2^k+1} = \sum_{i=1}^{k'} x^{2^{ik}} + \alpha\ \mathrm{Tr}(x), \qquad \alpha \in \{0,1\}, \tag{1}$$

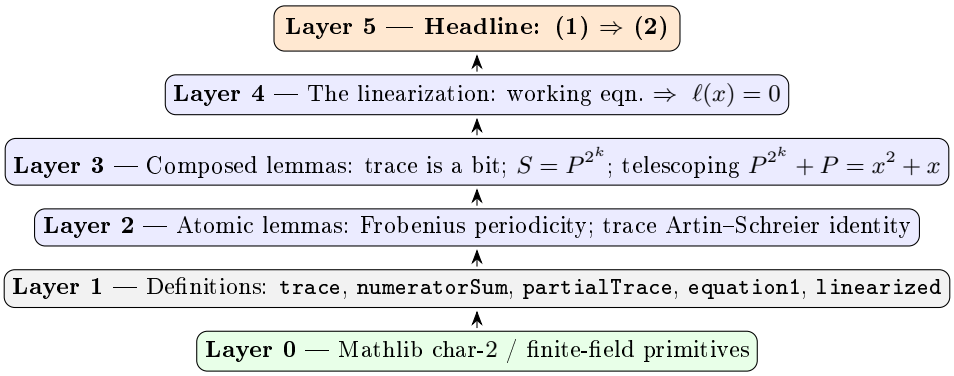
and its *linearized* consequence, equation (2), is $\ell(x) = 0$ where

$$\ell(x) = c^{2^k} x^{2^{2k}} + x^{2^k} + c\,x + 1. \tag{2}$$

Goal. Prove, in Lean, that for every nonzero x and every $\alpha \in \{0,1\}$, *(1) implies (2)*. In Lean this is one theorem:

```
theorem equation2_of_equation1 {n k k' α : ℕ} (hn : Fintype.card F = 2 ^ n)
    (hkk' : k * k' % n = 1) (hα : α = 0 ∨ α = 1) {c x : F} (hx : x ≠ 0)
    (h : equation1 n k k' α c x) : linearized k c x = 0
```

Plan (bottom-up). We build the proof in five layers. Each layer uses only the layers below it.



2 Layer 0 — the foundational ingredients

The whole development rests on a small, fixed set of facts about a finite field F of characteristic 2 with $|F| = 2^n$. These are the "atoms": everything else is assembled from them.

Fact	Mathematics	Mathlib name
Frobenius fixes F	$x^{\lvert F\rvert} = x$	`FiniteField.pow_card`
Freshman's dream	$(a+b)^2 = a^2 + b^2$	`add_pow_char`
iterated freshman	$(a+b)^{2^k} = a^{2^k} + b^{2^k}$	`add_pow_char_pow`
sum version	$(\sum a_i)^{2^k} = \sum a_i^{2^k}$	`sum_pow_char_pow`
$x + x = 0$	doubling is zero	`CharTwo.add_self_eq_zero`
$2 = 0$	characteristic two	`CharTwo.two_eq_zero`

Two structural consequences of char 2 drive every cancellation below:

$a + a = 0$ ("things that appear twice vanish"), $\qquad (a+b)^{2^k} = a^{2^k} + b^{2^k}$ ("2^k-th power is additive").

The Lean context carrying these is simply

```
variable {F : Type*} [Field F] [Fintype F] [CharP F 2]
```

3 Layer 1 — the definitions

Five definitions turn the paper's symbols into Lean objects (`DobbertinStep1/Defs.lean`). Nothing here is proved yet; these are the nouns of the story.

Symbol	Meaning	Lean
$\mathrm{Tr}(x) = \sum_{i<n} x^{2^i}$	absolute trace	`trace n x`
$S(x) = \sum_{i=1}^{k'} x^{2^{ik}}$	numerator sum	`numeratorSum k k' x`
$P(x) = \sum_{j<k'} x^{2^{jk}}$	partial trace	`partialTrace k k' x`
eqn. (1)	cleared equation	`equation1 n k k' α c x`
$\ell(x)$	linearized polynomial	`linearized k c x`

```
def trace (n : ℕ) (x : F) : F := ∑ i ∈ range n, x ^ (2 ^ i)

def numeratorSum (k k' : ℕ) (x : F) : F := ∑ i ∈ Icc 1 k', x ^ (2 ^ (i * k))

def partialTrace (k k' : ℕ) (x : F) : F := ∑ j ∈ range k', x ^ (2 ^ (j * k))

def equation1 (n k k' α : ℕ) (c x : F) : Prop :=
  c * x ^ (2 ^ k + 1) = numeratorSum k k' x + (α : F) * trace n x

def linearized (k : ℕ) (c x : F) : F :=
  c ^ (2 ^ k) * x ^ (2 ^ (2 * k)) + x ^ (2 ^ k) + c * x + 1
```

The key design choice is the pair S vs. P. The right-hand side of (1) is $S(x) = \sum_{i=1}^{k'} x^{2^{ik}}$; the *partial trace* $P(x) = \sum_{j=0}^{k'-1} x^{2^{jk}}$ is the same sum shifted down by one Frobenius step. Their relationship, $S = P^{2^k}$, is exactly what lets us "add the 2^k-th power of (1) to itself".

4 Layer 2 — atomic lemmas

These are the first proved facts; each rests *directly* on Layer 0.

4.1 Frobenius periodicity

Because $x^{2^n} = x^{|F|} = x$, the exponent 2^r only matters modulo n.

$$\boxed{x^{2^r} = x^{2^{\,r \bmod n}}} \qquad (\texttt{pow\_two\_pow\_mod})$$

```
lemma pow_two_pow_mod {n : ℕ} (hn : Fintype.card F = 2 ^ n) (x : F) (r : ℕ) :
    x ^ (2 ^ r) = x ^ (2 ^ (r % n)) := by
  conv_lhs => rw [show r = n * (r / n) + r % n from (Nat.div_add_mod r n).symm,
    pow_add, pow_mul]
  have step : ∀ j : ℕ, x ^ (2 ^ (n * j)) = x := by
    intro j
    induction j with
    | zero => simp
    | succ j ih => rw [Nat.mul_succ, pow_add, pow_mul, ← hn, FiniteField.pow_card, ih]
  rw [step]
```

Idea. Write $r = n\lfloor r/n \rfloor + (r \bmod n)$; the first part contributes $x^{2^{n\cdot j}} = x$ by iterating $x^{2^n} = x$ (j times).

4.2 The trace's Artin–Schreier identity

Squaring is additive, so squaring the telescoping sum $\mathrm{Tr}(x) = \sum_{i<n} x^{2^i}$ collapses everything except the two ends:

$$\boxed{\mathrm{Tr}(x)^2 + \mathrm{Tr}(x) = x^{2^n} + x} \qquad (\texttt{trace\_sq\_add\_self}).$$

```
lemma trace_sq_add_self (n : ℕ) (x : F) :
    trace n x ^ 2 + trace n x = x ^ (2 ^ n) + x := by
  induction n with
  | zero => simp [trace, CharTwo.add_self_eq_zero]
  | succ n ih => ...  -- peel off the top summand, use (a+b)²=a²+b², cancel in char 2
```

Idea. Induct on n. The step splits $\mathrm{Tr}_{n+1} = \mathrm{Tr}_n + x^{2^n}$; squaring is additive, so $\mathrm{Tr}_{n+1}^2 = \mathrm{Tr}_n^2 + x^{2^{n+1}}$; the induction hypothesis and a char-2 cancellation of the repeated x^{2^n} finish it. (This lemma needs no finiteness of F — it is pure characteristic-2 algebra, hence `omit [Fintype F]`.)

5 Layer 3 — composed lemmas

Now the two independent strands of the proof appear: the *trace bit* and the *telescoping*.

5.1 Strand A: the trace is a bit

Specialising $x^{2^n} = x$ in the Artin–Schreier identity gives idempotence, and idempotence over a field forces $\mathrm{Tr}(x) \in \{0, 1\}$.

$$\mathrm{Tr}(x)^2 = \mathrm{Tr}(x) \underset{\mathrm{Tr}(\mathrm{Tr}-1)=0}{\Longrightarrow} \boxed{\mathrm{Tr}(x) = 0 \ \vee \ \mathrm{Tr}(x) = 1}$$

```
lemma trace_sq {n : ℕ} (hn : Fintype.card F = 2 ^ n) (x : F) :
    trace n x ^ 2 = trace n x
lemma trace_eq_zero_or_one {n : ℕ} (hn : Fintype.card F = 2 ^ n) (x : F) :
    trace n x = 0 ∨ trace n x = 1
```

This is the "$= 0$ or 1" input Dobbertin uses: it lets us collapse the term $\alpha\,\mathrm{Tr}(x)$ into a single bit $\varepsilon \in \{0, 1\}$.

5.2 Strand B: telescoping the partial trace

First, the shift relation (S is the Frobenius image of P):

$$S(x) = P(x)^{2^k} \qquad (\texttt{numeratorSum\_eq\_partialTrace\_frob}).$$

```
lemma numeratorSum_eq_partialTrace_frob (k k' : ℕ) (x : F) :
    numeratorSum k k' x = partialTrace k k' x ^ (2 ^ k)
```

Then the char-2 telescoping of the linearized polynomial P: adding P^{2^k} to P makes every interior term appear twice, leaving only the ends,

$$P(x)^{2^k} + P(x) = x^{2^{k'k}} + x \qquad (\texttt{partialTrace\_telescope\_raw}).$$

```
lemma partialTrace_telescope_raw (k k' : ℕ) (x : F) :
    partialTrace k k' x ^ (2 ^ k) + partialTrace k k' x
      = x ^ (2 ^ (k' * k)) + x
```

Finally, feed in Frobenius periodicity (Section 4.1): since $k\,k' \equiv 1 \pmod{n}$, the exponent $2^{k'k}$ collapses to $2^1 = 2$, giving the Artin–Schreier form

$$\boxed{P(x)^{2^k} + P(x) = x^2 + x} \qquad (\texttt{partialTrace\_telescope}).$$

```
lemma partialTrace_telescope {n k k' : ℕ} (hn : Fintype.card F = 2 ^ n)
    (hkk' : k * k' % n = 1) (x : F) :
    partialTrace k k' x ^ (2 ^ k) + partialTrace k k' x = x ^ 2 + x := by
  rw [partialTrace_telescope_raw]
  have : x ^ (2 ^ (k' * k)) = x ^ 2 := by
    rw [pow_two_pow_mod hn x (k' * k), Nat.mul_comm k' k, hkk', pow_one]
  rw [this]
```

6 Layer 4 — the linearization

This is the algebraic heart: "adding the 2^k-th power of the equation to itself". Assume $x \neq 0$ and that x solves the *working* form of (1) with the trace term already collapsed to a bit $\varepsilon \in \{0, 1\}$:

$$S(x) + \varepsilon = c\, x^{2^k+1}. \tag{1′}$$

```
lemma linearized_eq_zero_of_solution {n k k' : ℕ} (hn : Fintype.card F = 2 ^ n)
    (hkk' : k * k' % n = 1) {ε : F} (hε : ε = 0 ∨ ε = 1) {c x : F} (hx : x ≠ 0)
    (hsol : numeratorSum k k' x + ε = c * x ^ (2 ^ k + 1)) :
    linearized k c x = 0
```

Derivation. Write $S = P^{2^k}$ and $T = P^{2^k} + P = x^2 + x$.

1. **Solve (1′) for P.** From $T = x^2 + x$ and $P^{2^k} = S = c\, x^{2^k+1} + \varepsilon$,
$$P = (x^2 + x) + c\, x^{2^k+1} + \varepsilon.$$
2. **Raise to the 2^k-th power** (additive, and $\varepsilon^{2^k} = \varepsilon$ since ε is a bit):
$$S = P^{2^k} = (x^2)^{2^k} + x^{2^k} + \left(c\, x^{2^k+1}\right)^{2^k} + \varepsilon.$$
3. **Cancel** ε against (1′) to reach the core relation between powers of x:
$$c\, x^{2^k+1} = (x^2)^{2^k} + x^{2^k} + \left(c\, x^{2^k+1}\right)^{2^k}.$$
4. **Reduce the exponents.** Using $(x^2)^{2^k} = x^{2^k} \cdot x^{2^k}$ and $(2^k + 1)2^k = 2^{2k} + 2^k$,
$$\left(c\, x^{2^k+1}\right)^{2^k} = c^{2^k}\left(x^{2^{2k}} \cdot x^{2^k}\right).$$
5. **Divide by $x^{2^k} \neq 0$.** Every surviving term carries a factor x^{2^k}; cancelling it turns the core relation into $c\, x = x^{2^k} + 1 + c^{2^k} x^{2^{2k}}$, i.e.
$$\ell(x) = c^{2^k} x^{2^{2k}} + x^{2^k} + c\, x + 1 = 0. \qquad \blacksquare$$

The five steps correspond one-to-one to the `have`s in the Lean proof (`hP_sub`, `hS_pow`, `h_core`, `e1`/`e2`, the final `mul_left_cancel`$_0$); the character-2 cancellations are discharged by `grind +ring` and `linear_combinatio`

Remark (why $x \neq 0$ is essential)**.** *At $x = 0$ the cleared equation (1) reads $0 = 0$ and holds* vacuously*, yet $\ell(0) = 1 \neq 0$. Step 5 divides by x^{2^k}, which is exactly where $x = 0$ is excluded — faithfully matching Dobbertin, who works with the nonzero solutions.*

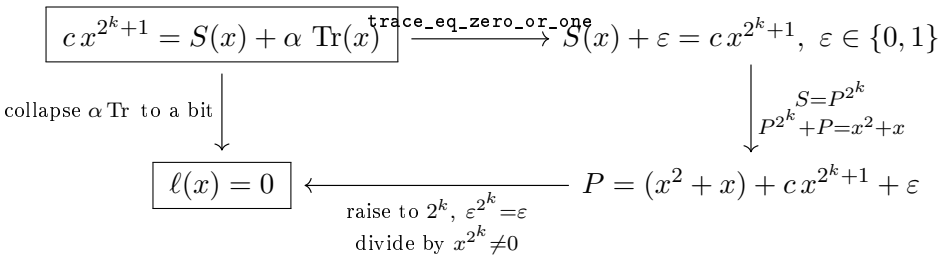
7 Layer 5 — the headline

The top theorem simply glues the two strands: use `trace_eq_zero_or_one` (Strand A) to collapse $\alpha\ \mathrm{Tr}(x)$ to a bit ε, then apply `linearized_eq_zero_of_solution` (Strand B + Layer 4).

```
theorem equation2_of_equation1 {n k k' α : ℕ} (hn : Fintype.card F = 2 ^ n)
    (hkk' : k * k' % n = 1) (hα : α = 0 ∨ α = 1) {c x : F} (hx : x ≠ 0)
    (h : equation1 n k k' α c x) :
    linearized k c x = 0 := by
  have hbit : (α : F) * trace n x = 0 ∨ (α : F) * trace n x = 1 := by
    have hαcast : (α : F) = 0 ∨ (α : F) = 1 := by rcases hα with rfl | rfl <;> simp
    rcases hαcast with h0 | h1
    · exact Or.inl (by rw [h0, zero_mul])
    · rw [h1, one_mul]; exact trace_eq_zero_or_one hn x
  exact linearized_eq_zero_of_solution hn hkk' hbit hx h.symm
```

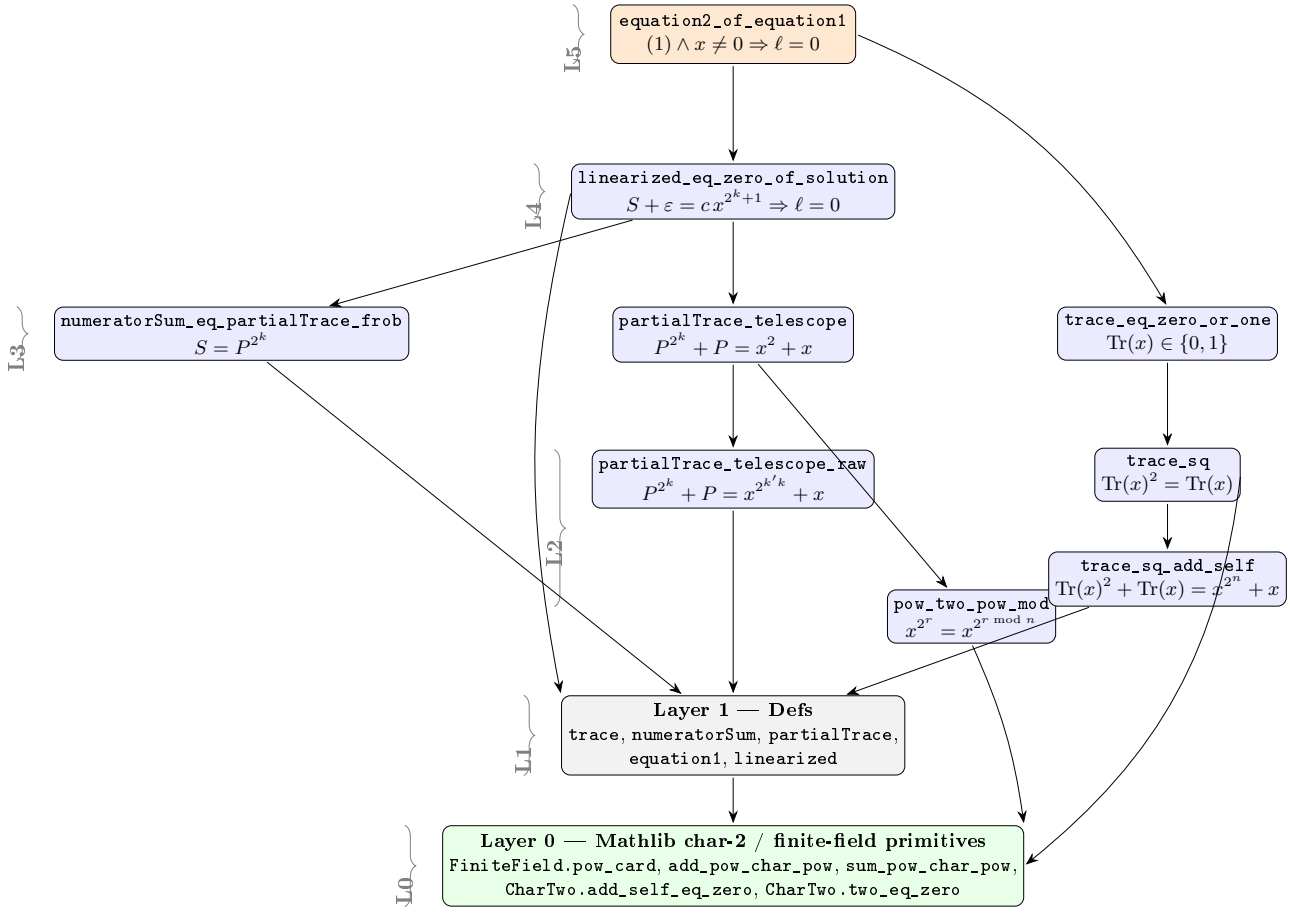
8 The derivation as a commutative diagram

Writing $\varepsilon = \alpha\ \mathrm{Tr}(x) \in \{0,1\}$, the two inputs (trace bit and telescoping) combine into the single implication:



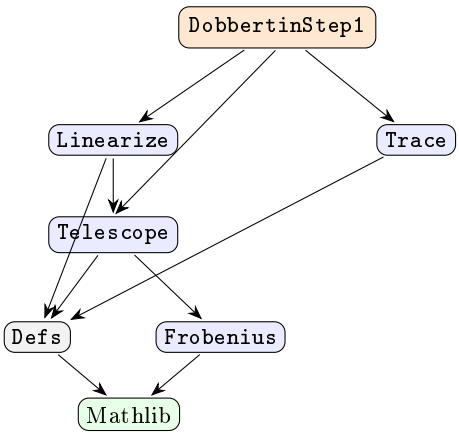
9 The full compositional build

The diagram below shows *everything at once*: how each declaration is assembled from the ones beneath it, organised by the five layers. An edge $A \to B$ means "A is built using B". The two vertical strands (trace bit on the right, telescoping on the left) meet only at the headline.

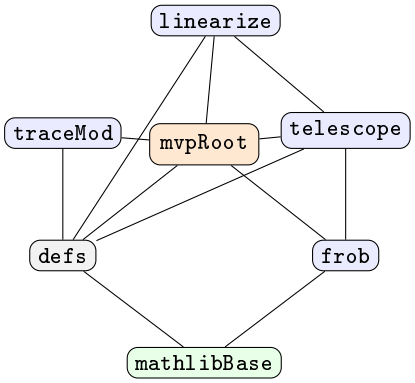


10 Module structure and the proof space

The declarations live in six modules over a common Mathlib base. The *directed* dependency DAG (left) is transcribed *verbatim* into a Lean relation `dep`; the induced *undirected* `SimpleGraph depGraph` (right) is proved **connected** in `DobbertinStep1/ProofSpace.lean`, certifying that no component is orphaned.



directed DAG = Lean `dep`



undirected `depGraph` = `SimpleGraph.fromRel dep`

```
theorem depGraph_connected : depGraph.Connected
```

Soundness

Every declaration in `DobbertinStep1` is `sorry`-free; `lake build` succeeds, and the headline `equation2_of_equatio` depends only on the standard axioms `propext`, `Classical.choice`, `Quot.sound`.